Campus Hub

Project Report

Muhammad Shadman Sakeeb

ID: 24101172

Sabab Muhammad Sarar

ID: 24101443

CSE370L: Database Systems Lab

Section: 18

May 7, 2026

Overview

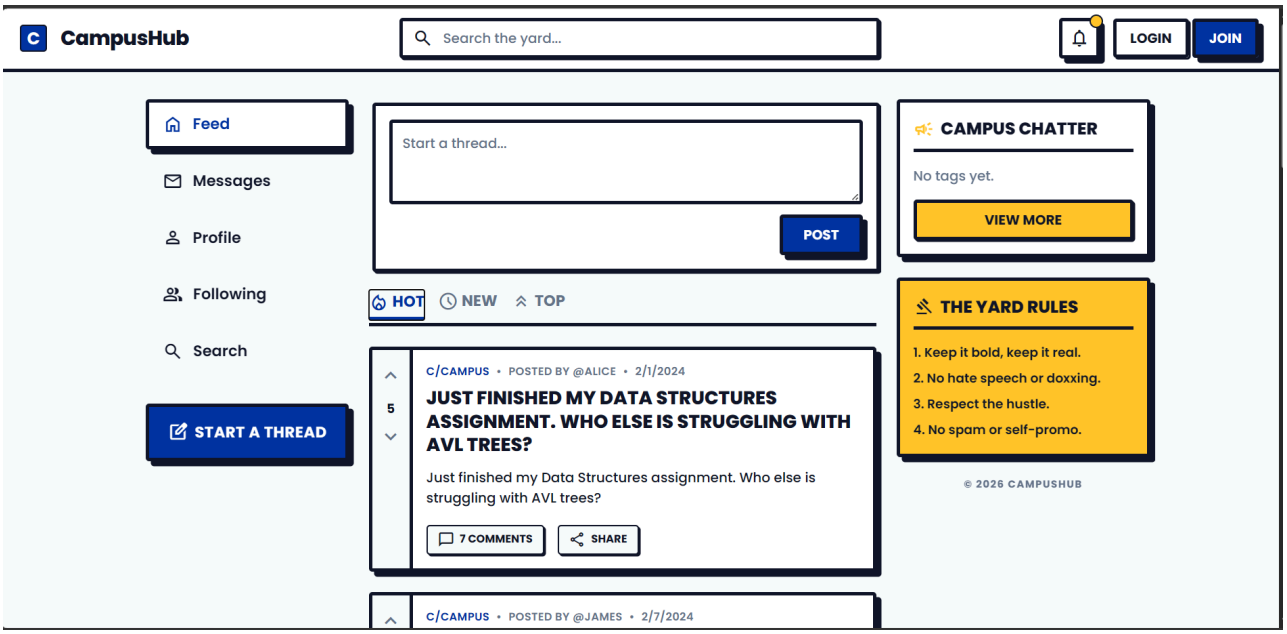
Campus Hub is a campus-focused social platform designed to help students, faculty, and staff connect with the community. It brings together a campus-wide feed, profiles, and discovery tools so people can share updates and stay involved.

This report summarizes the core user-facing features in plain language. It also explains, at a high level, how each feature uses the database behind the scenes.

Platform at a Glance

Campus Hub centers on quick sharing and simple discovery. Users sign in, post updates, and engage with classmates or colleagues. The platform encourages lightweight interactions such as comments, replies, and votes, while also helping users build their campus network.

The experience is designed to feel familiar: a feed for updates, profiles for identity, and search for discovery. Behind the scenes, the system stores users, posts, and relationships in a structured database so that each page can load quickly and reliably.



Data Model Overview

The database model organizes how Campus Hub stores users, posts, comments, votes, and follow relationships. For a database course, these visuals provide a clear, structural view of the system's data entities and how they relate.

The EER diagram shows entity relationships at a conceptual level, while the schema diagram highlights tables, keys, and relationships in the relational model.

EER Diagram:

Diagram source: docs/eer.pdf

Relational Schema:

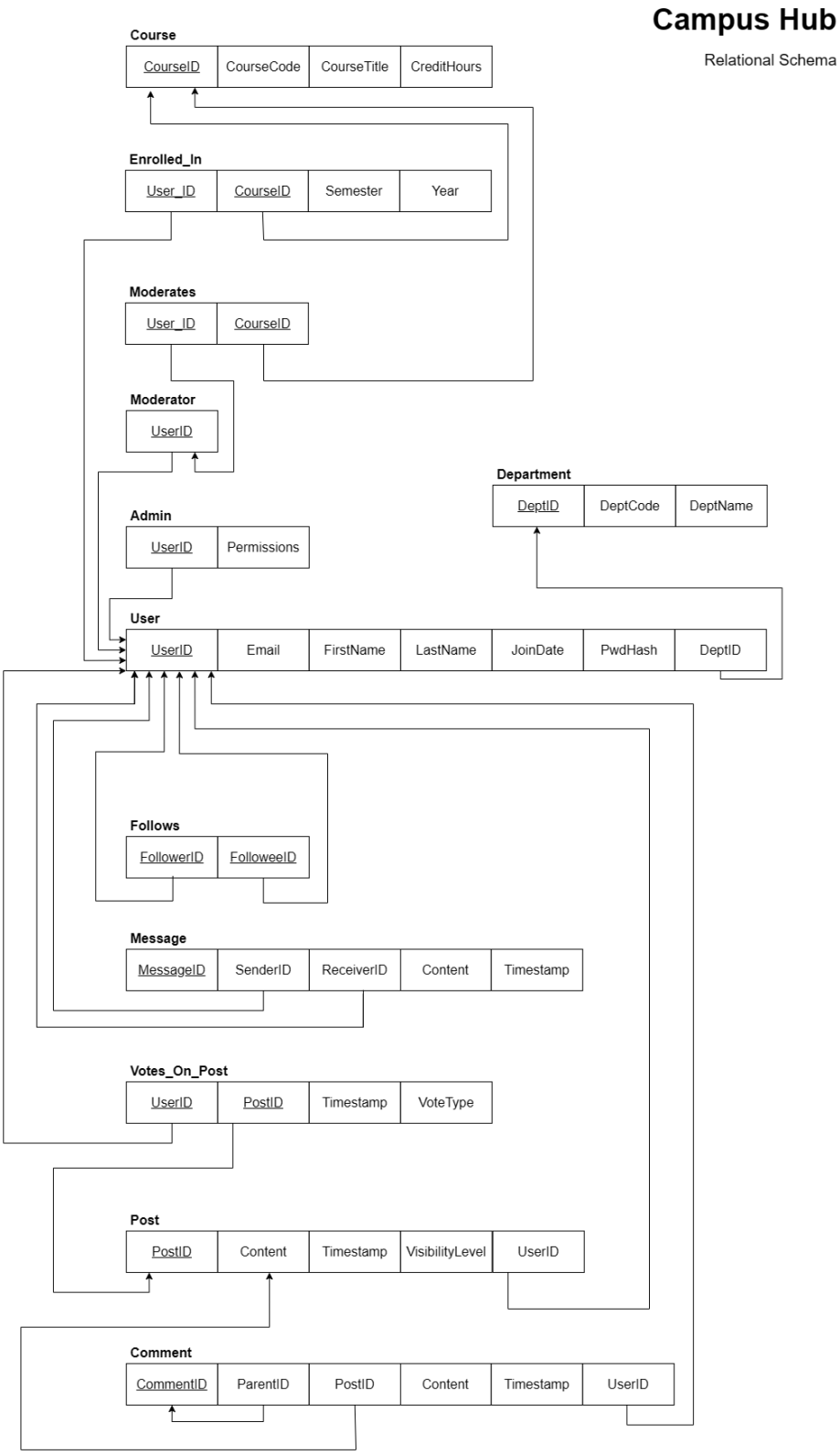


Diagram source: docs/schema.pdf

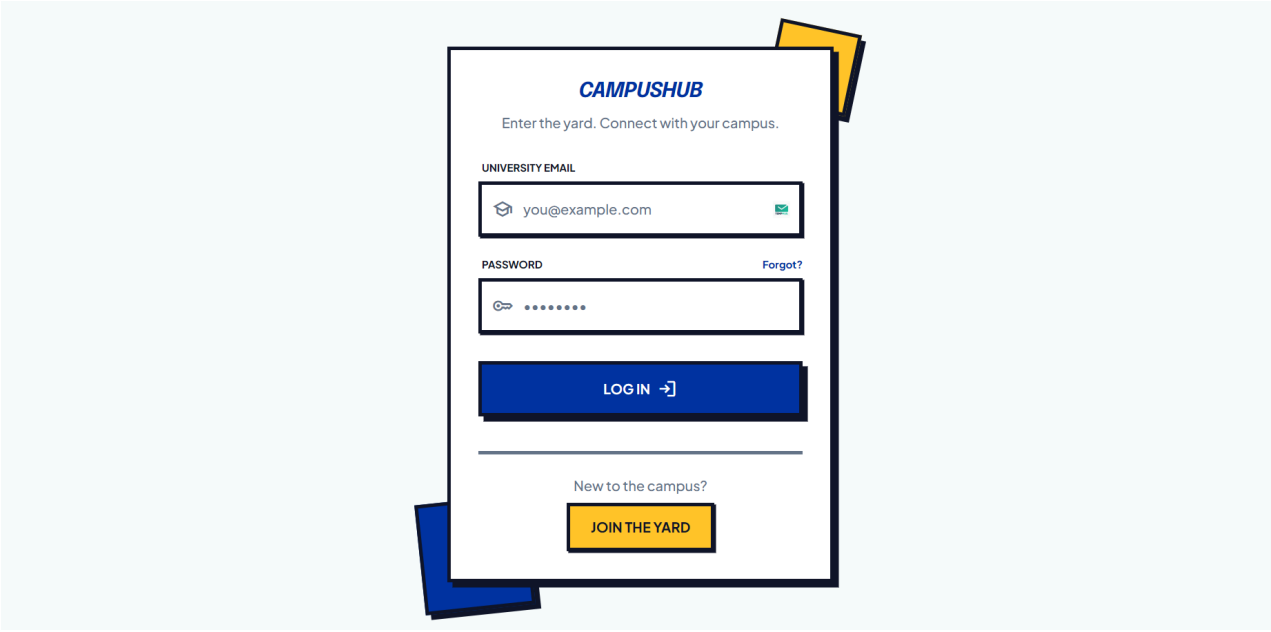
Feature Overview

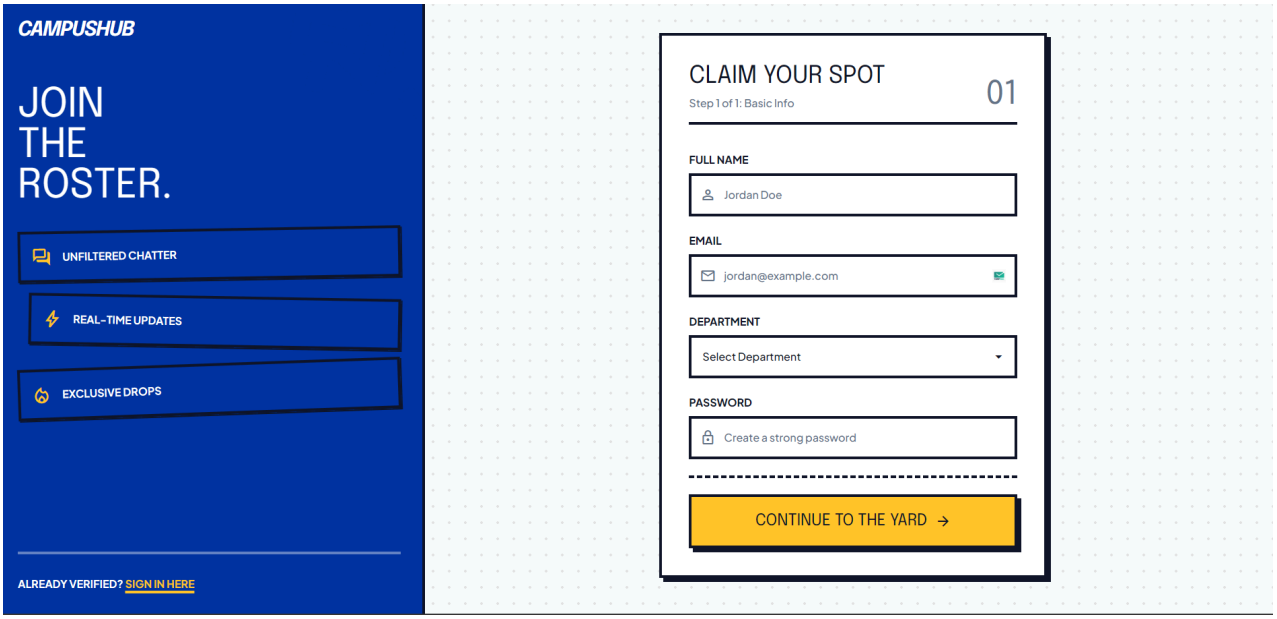
#	Feature	Description	CRUD Operations
1	Authentication	Users can register a new account, login with email and password, and logout.	Create account (Register), Read session (Login), Delete session (Logout)
2	User Profile	Users can view any profile on the platform and update their own personal information.	Read profile, Update profile info (name, bio, department)
3	Posts	Users can create text posts visible on the campus-wide feed, browse all posts, and delete their own.	Create post, Read posts, Delete post
4	Comments	Users can add comments to any post, view all comments under a post, and delete their own comments.	Create comment, Read comments on post, Delete comment
5	Replies	Users can reply directly to an existing comment, forming a nested thread. Replies can be viewed and deleted.	Create reply, Read replies under comment, Delete reply
6	Voting	Users can upvote or downvote any post. A user can change or remove their vote at any time.	Create vote, Update vote type (up/down), Delete vote, Read vote count
7	Follow System	Users can follow or unfollow other users and view their own followers and following lists.	Create follow, Delete follow (unfollow), Read followers and following
8	Direct Messaging (Excluded)	Users can send real-time direct messages to any other user and view their full conversation history.	Create message, Read conversation history
9	Search	Users can search for other users by name or department to find and visit their profiles.	Read users by search query

Feature 1: Authentication

Authentication allows users to create an account, sign in, and safely access the platform. This is the first touchpoint for a new user and should feel welcoming and straightforward.Once authenticated, the user can access the feed, profiles, and campus community features.

Register screen submits a POST request to /api/auth/register with name, email, password, and optional department. Backend checks the User table for an existing email (SELECT). If none, it hashes the password and inserts a new row into User (INSERT). Login screen submits a POST request to /api/auth/login with email and password. Backend SELECTs UserID and PwdHash, verifies the password, and returns a JWT token to the client.





Feature 2: User Profile

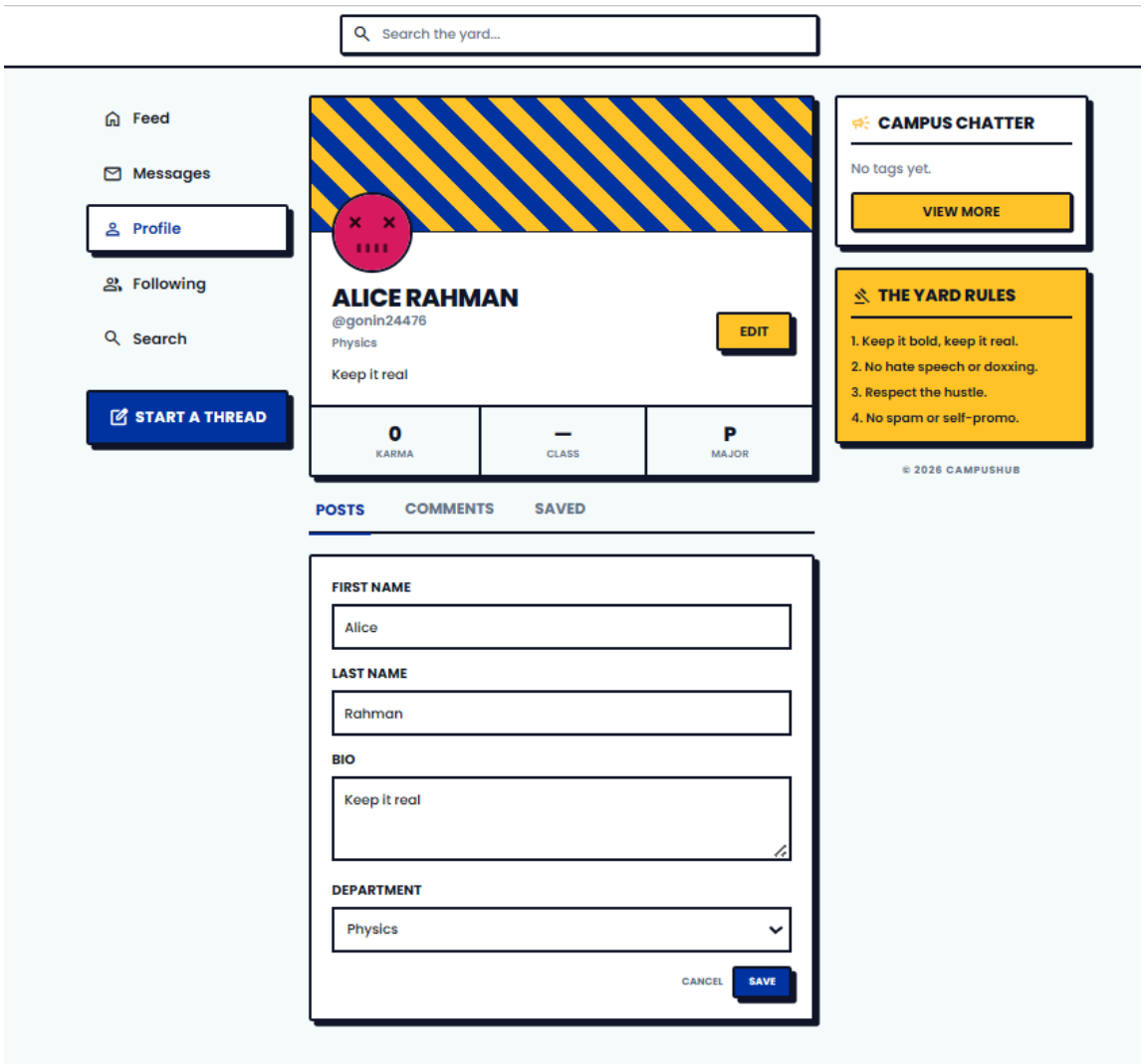
Profiles are the personal identity spaces within Campus Hub. Each user can view and update their name, bio, and department. This helps others understand who they are interacting with and builds a sense of community.

The profile page also serves as a snapshot of a user's activity and presence on the platform. It keeps the experience friendly and context-rich without requiring technical steps.

Profile page loads data with a GET request to /api/users/{user_id}. The backend SELECTs from User and joins Department for the department name.

If the viewer is the owner, a PATCH request to /api/users/me updates profile fields. The backend runs an UPDATE on the User table.

The profile page also loads the user's posts via GET /api/posts?user_id=... which SELECTs posts tied to that user.



Feature 3: Posts (Campus Feed)

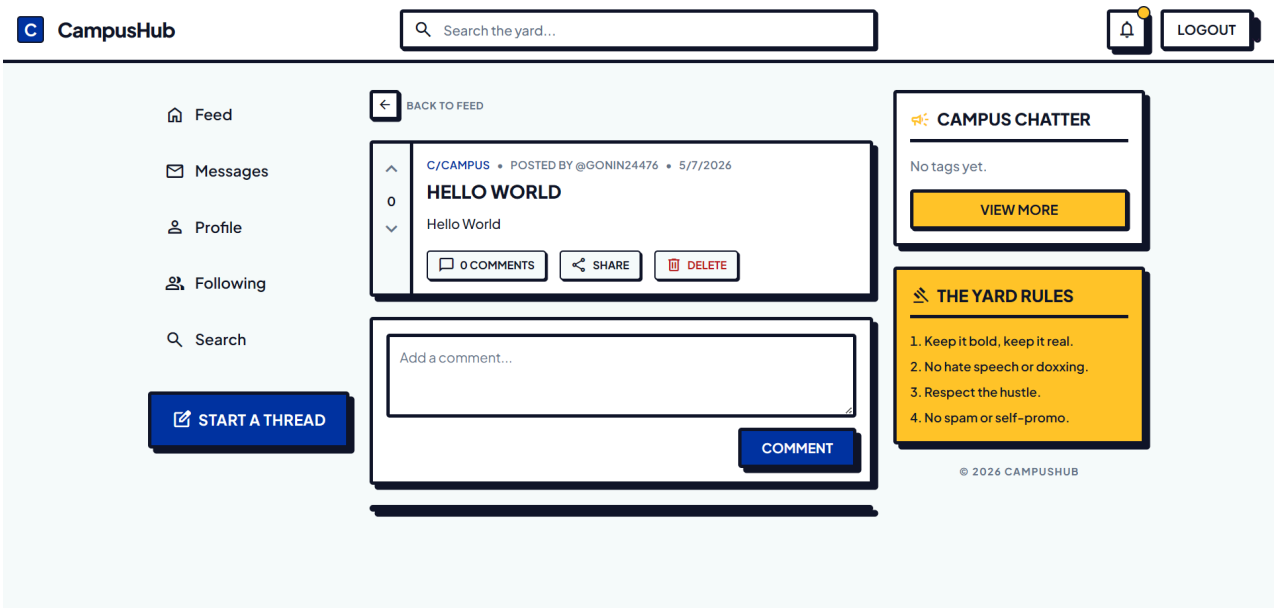
Posts are the main way users share updates, announcements, and ideas. The campus feed brings these posts together so users can quickly browse what is happening across the community.

Creating a post is designed to be quick and familiar, with a simple composer and a clear submit action. The feed then displays posts in a readable, scannable format that supports daily use.

Feed loads with GET /api/posts?sort=... which SELECTs from Post, joins User, and computes vote score and comment count using aggregate queries.

Creating a post sends POST /api/posts with content and visibility. Backend INSERTs a new row into Post with the current UserID.

Deleting a post sends DELETE /api/posts/{post_id}. Backend checks ownership (SELECT) and removes the row (DELETE).



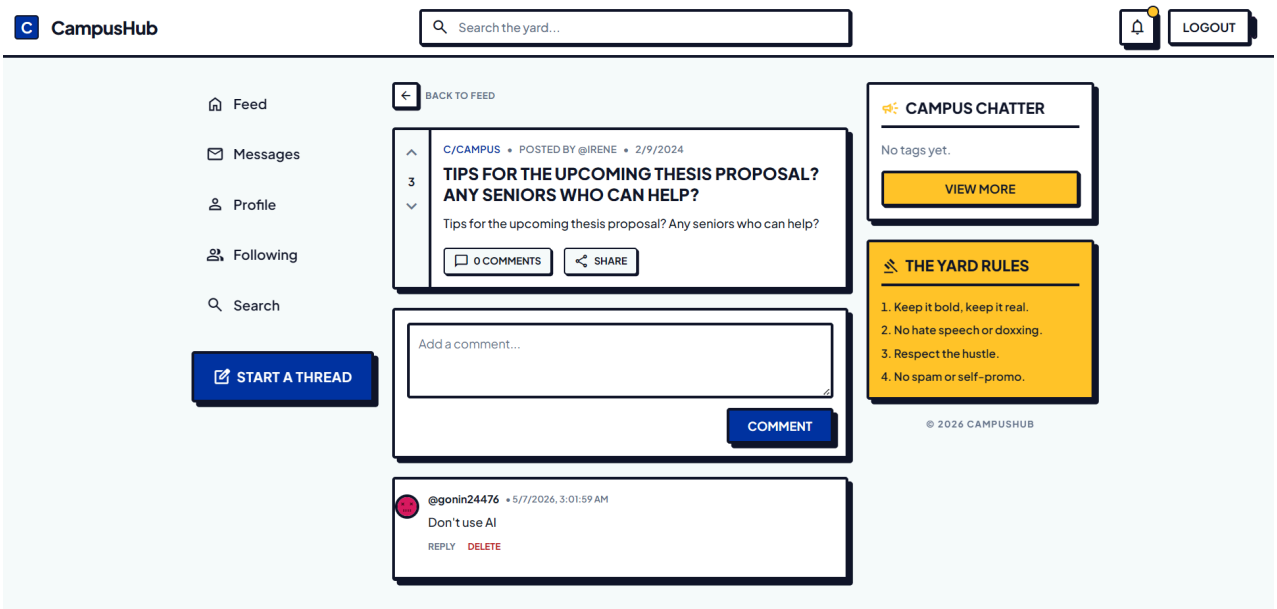
Feature 4: Comments

Comments let users respond to posts in a focused, lightweight way. They enable quick conversation without leaving the post. This keeps discussions organized and easy to follow.

Users can add new comments and review existing ones directly within a post's detail view. The result is a clear, threaded conversation around each update.

Post detail page loads comments with GET /api/comments/posts/{post_id}. Backend SELECTs from Comment and joins User to show usernames.

Submitting a new comment sends POST /api/comments/posts/{post_id}. Backend verifies the Post exists (SELECT) and INSERTs a Comment row with ParentID = NULL.



Feature 5: Replies

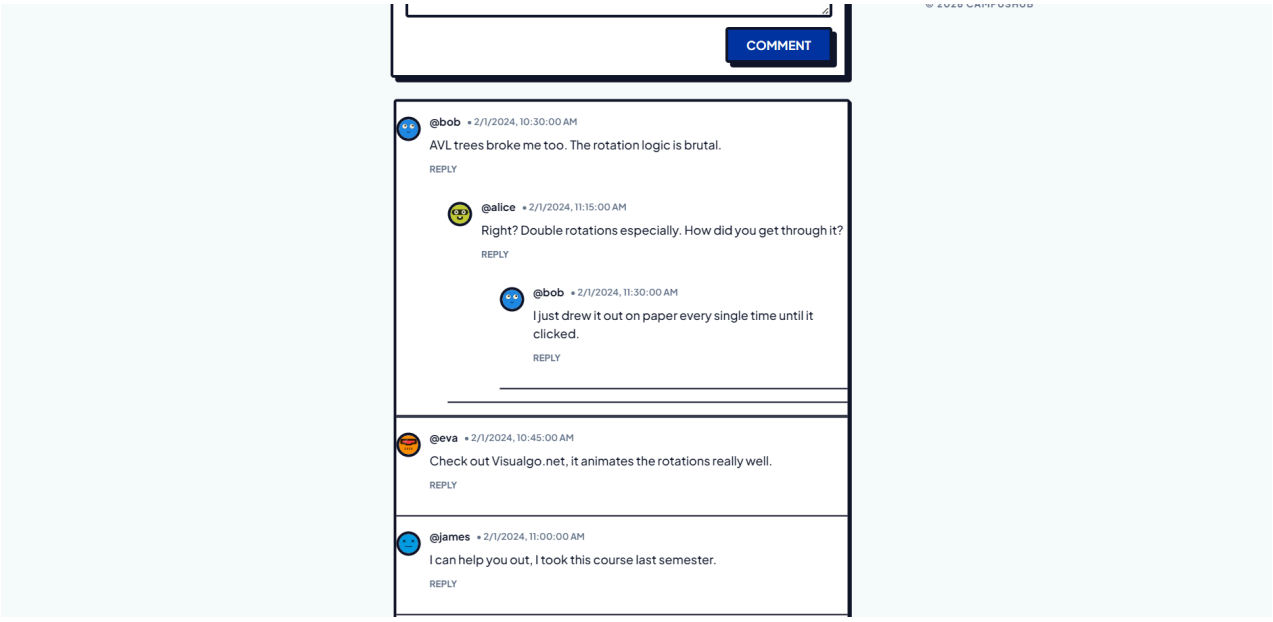
Replies build on comments by allowing nested responses. This helps users reply to specific remarks without cluttering the main thread.

The nested format makes it easier to follow back-and-forth discussion within a post. It encourages more meaningful interaction while keeping the overall conversation tidy.

Reply action sends POST /api/comments/{comment_id}/replies with reply content.

Backend SELECTs the parent comment to find its PostID, then INSERTs a Comment row with ParentID set to the parent comment.

The UI builds a threaded view by grouping comments with ParentID values under their parent comment.



Feature 6: Voting

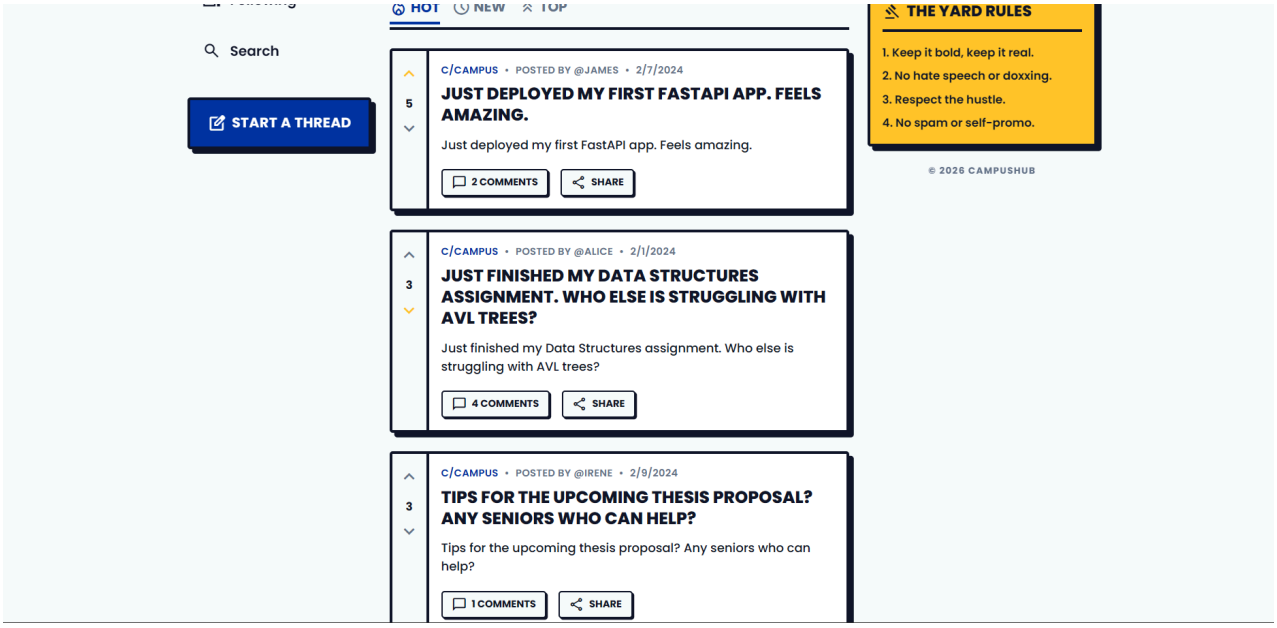
Voting allows users to quickly show support or disagreement with a post. This creates a simple, familiar signal of what the community values.

Users can upvote or downvote and adjust their choice at any time. The interface keeps these actions visible but unobtrusive, so the feed remains easy to read.

Voting sends POST /api/votes/posts/{post_id} with vote_type. Backend checks the post exists (SELECT).

If a vote already exists in Votes_On_Post, the backend UPDATEs it; otherwise it INSERTs a new row.

Removing a vote sends DELETE /api/votes/posts/{post_id}, which deletes the row from Votes_On_Post.



Feature 7: Follow System

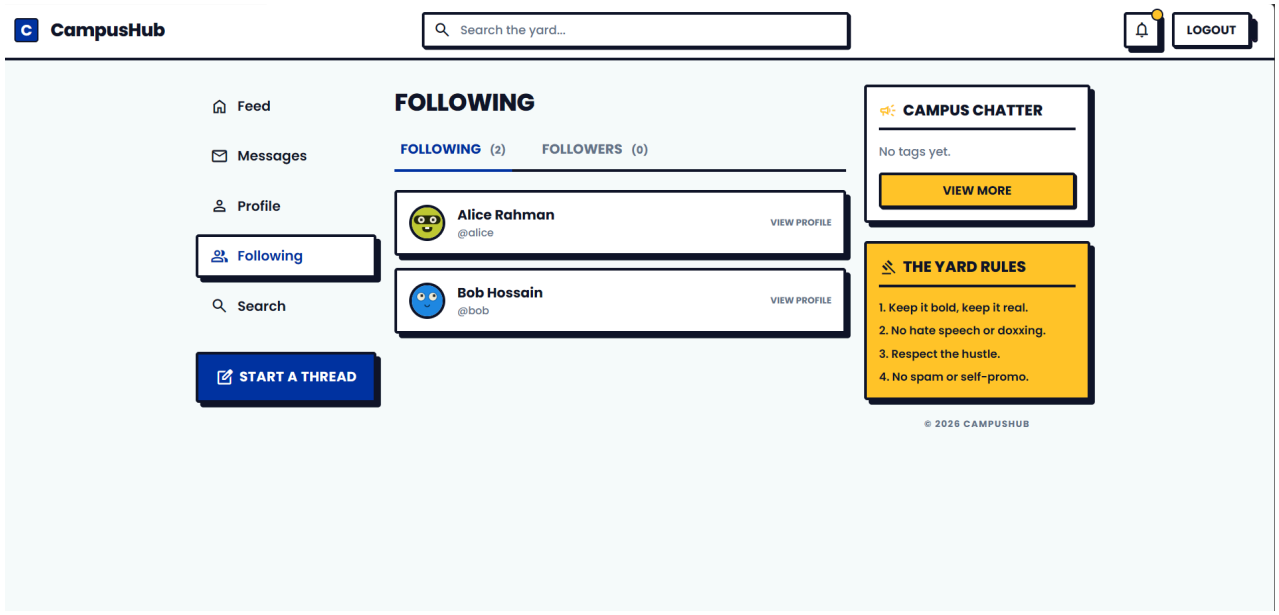
Following lets users build a personalized network. By following classmates or colleagues, users can keep track of updates from people they care about most.

The follow system makes it easy to see who you follow and who follows you, supporting ongoing campus connections.

Follow action sends POST /api/follows/users/{user_id}. Backend validates the user exists (SELECT) and INSERTs a row into Follows.

Unfollow sends DELETE /api/follows/users/{user_id}, which removes the matching row from Follows.

Followers and following lists load from GET /api/users/{user_id}/followers and /following, which JOIN Follows with User.



Feature 8: Search

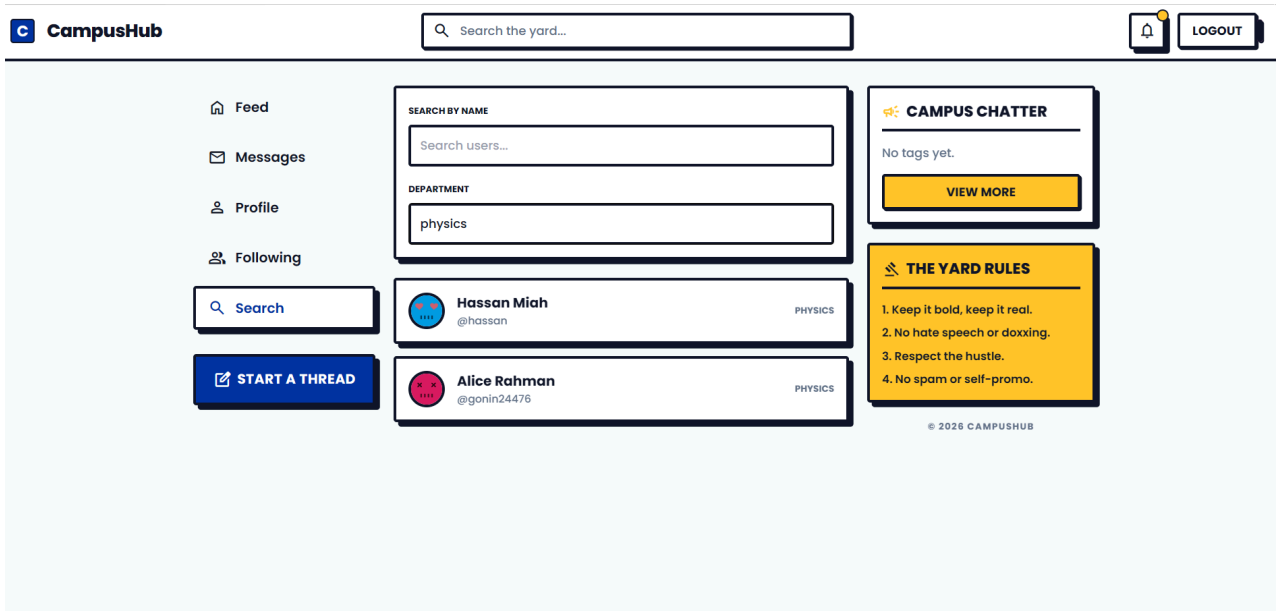
Search helps users quickly find other people in the campus community. It supports searching by name or department, which makes discovery easy even for large campuses.

The search experience is simple: enter a query, review results, and navigate to the profile you need.

Search sends GET /api/search/users?query=...&department=... .

Backend SELECTs from User and LEFT JOINs Department, using LIKE queries against names, usernames, and department names.

Results return as a list of users for the UI to display.



Conclusion

Campus Hub focuses on simple, everyday interactions that help campus communities stay connected. By combining a campus-wide feed with profiles, comments, voting, and search, it provides a complete yet approachable experience.

This report presented the core features in non-technical terms while also outlining the main database operations supporting each feature. With these materials, the project can be communicated clearly to a general audience and assessed within a database course context.